Week 21 (Days 141-147)

Status: Days 141-147 (Month 5, MLOps and ML in production) are written in full. Every number comes from `code/week-21/day141_*.py` to `day147_*.py`, each run on this box, with logs in `code/week-21/out/`. Shared helpers are in `code/week-21/_mlops.py`; the project service, tests, model card, Dockerfile and CI file are in `code/week-21/project/`.

Installs verified this week (pip, into the course venv): mlflow-skinny 3.17.2, fastapi 0.162.0, uvicorn 0.54.0, pydantic 2.14.1, httpx 0.28.1, onnx 1.23.2, onnxruntime 1.31.1, onnxscript 0.7.2, pytest 9.2.0. These are pinned in `requirements.txt`. Already present: torch 2.15.0+cpu, scikit-learn 1.9.1, scipy 1.18.1, numpy 2.5.4.

Not available, and labelled wherever it appears: - There is **no Docker** on the box. `project/Dockerfile` and `project/ci/github-actions-ci.yml` are written but **UNTESTED**: never built, never run. - MLflow's file-store backend is in "maintenance mode" in 3.17 and raises an error unless `MLFLOW_ALLOW_FILE_STORE=true` is set. Its sqlite backend needs SQLAlchemy, which mlflow-skinny does not include. Day 143 used the file store with that opt-out.

Compute note. The model is a 30-32-32-2 MLP on scikit-learn's bundled Wisconsin breast-cancer table (569 rows), so every script runs in seconds on one CPU thread. A GPU would change nothing here. What it changes in real production is the serving side for large models: batching and GPU memory dominate latency and cost, and Day 146's cost arithmetic would be per GPU-hour with very different throughput numbers. The load tests ran the client and server on the same machine, so they compete for CPU. The latency numbers are for localhost and include no network.

Sources (opened and checked): DeepLearning.AI / Coursera "Introduction to Machine Learning in Production", https://www.coursera.org/learn/introduction-to-machine-learning-in-production (lifecycle ideas below are summarised in my own words, not copied); DVC docs, https://dvc.org/doc; MLflow docs, https://mlflow.org/docs/latest/; FastAPI docs, https://fastapi.tiangolo.com/; ONNX Runtime docs, https://onnxruntime.ai/docs/; Dockerfile reference, https://docs.docker.com/reference/dockerfile/; GitHub Actions docs, https://docs.github.com/en/actions.

Day 141: The ML project lifecycle and scoping with numbers

Review today: Day 140 (+1), Day 138 (+3), Day 134 (+7), Day 120 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The lifecycle, in my own words. A production ML project loops through four stages, and you will go round more than once: 1. **Scoping:** decide what problem is worth solving, what "good enough" means, and what it is worth. 2. **Data:** define labels and inputs, collect, clean, validate and version them. 3. **Modelling:** train, do error analysis, iterate. Usually the data changes more than the code. 4. **Deployment and monitoring:** serve it, watch inputs and outputs, and feed what you learn back into data and modelling.

The common lesson from practitioners (Andrew Ng's "ML in Production" course is the best-known framing) is that the model code is a small part of the system. Most of the work is in data definition, deployment plumbing and feedback loops.

Scoping questions to answer with numbers before building anything: - *What is the baseline?* Do nothing (majority class), a one-line rule, a simple linear model, and human-level performance if you can measure it. If a rule gets 80% and your target is 82%, ML may not be worth its maintenance cost. - *What does an error cost?* Errors are rarely symmetric. In a screening task a missed positive can cost far more than a false alarm. Pick the metric and the decision threshold from those costs, not from accuracy. - *Will more data help?* A learning curve (accuracy vs training-set size) tells you whether to spend on labelling or on modelling. A flat curve means more of the same data won't help. - *Where does it fail?* Slice-level error analysis (accuracy by subgroup) points to which data to collect next, and is the start of the error-analysis loop we close on Day 145.

Today's running example. We use the Wisconsin breast-cancer table bundled with scikit-learn: 30 numeric features computed from cell images, label malignant (0) or benign (1). It is small, offline and well-known. It is a teaching dataset, not a clinical tool, and the model card on Day 147 says so.

Block 2: Code walk-through (1h)

`day141_scoping.py` uses a stratified 75/25 split (seed 0). It evaluates the majority class, a one-feature rule (worst radius below the training median -> benign), standardised logistic regression and our MLP (`_mlops.MLP`). The MLP keeps its normalisation constants *inside* the model as buffers, so serving cannot forget to normalise. The script then draws a learning curve over 3 seeds per training-set size, applies a 10:1 cost to missed malignant cases versus false alarms, and computes test accuracy by tertiles of "mean area".

Verified (out/day141_scoping.log): - Data: 426 train / 143 test rows, 30 features, benign rate 0.627. - Baselines: majority 0.629; one-feature rule 0.804; logistic regression 0.958; MLP 0.965 (trained in 0.9 s). - Learning curve (mean of 3 seeds): 10% (42 rows) 0.914; 25% (106 rows) 0.946; 50% (213 rows) 0.944; 100% (426 rows) 0.953. The curve is nearly flat after about 100 rows. The 100% value is below the single-seed 0.965 because of seed variance. - Cost (10 x missed + false alarms):

threshold	missed malignant	false alarms	cost
0.5	3	2	32
0.8	2	4	**24**
0.95	2	5	25

- Slices by mean area: small 1.000 (0 errors of 48), medium 0.915 (4 errors of 47), large 0.979 (1 error of 48).

Readings: - Logistic regression is one test error behind the MLP. On 143 rows, one error is 0.007 of accuracy, so 0.958 vs 0.965 is a single example. Scoping says to ship the simpler model unless the MLP buys something measurable. - The flat learning curve says that buying more of the same data is a poor investment. Better labels or new features would be a better use of the money. - Errors concentrate in the medium-area slice, which is the borderline region. That is where to look and where to collect more data.

Block 3: Exercises (1h30)

1. Add a "human-level" proxy: train on half the features and call that a weaker expert. How do you decide when to stop improving?
2. Change the cost ratio to 50:1 and re-pick the threshold. What's the cost at 0.5?
3. Add confidence intervals to the learning curve using more seeds. Is 0.944 vs 0.946 a real difference?
4. Write a one-page scoping document for this model: problem, users, baseline, metric, target, cost of errors, data plan, risks.

Block 4: Self-check (30 min)

1. Why compute a one-line rule baseline? *It sets the bar ML must clear to justify its cost: here 0.804.*
2. What does a flat learning curve tell you? *More of the same data won't help much. Change the data quality, features or model instead.*
3. Why did the best threshold move from 0.5 to 0.8? *Missed malignant cases cost 10x, so trading 2 extra false alarms for 1 fewer miss lowered the cost from 32 to 24.*
4. Why prefer logistic regression here despite 0.958 < 0.965? *The gap is one test example, and the simpler model is cheaper to serve, explain and maintain.*
5. What's the next data action from the slice table? *Collect and review borderline medium-area cases, where 4 of the 5 errors sit.*

Block 5: Reflection (15 min)

Write the one number you would need to see before approving this project for deployment, and who should set it.

Extra notes. Scoping is also where you decide *not* to do ML. A rule at 0.804 with zero maintenance can beat a model at 0.96 when errors are cheap and the data keep changing.

Going deeper. Read the scoping module of the Coursera course in the header. Then write the same scoping document for a Week 17 RAG feature. Its "baseline" is keyword search.

Interview-style question (15 min). "Your PM wants 99% accuracy. How do you respond?" Ask what the errors cost and to whom, what the baseline and human-level numbers are, and whether accuracy is even the right metric. Show a learning curve to estimate what more data would buy. Propose a threshold that minimises cost, plus a human-review band for uncertain cases.

Common mistakes. No baseline. Optimising accuracy when errors are asymmetric. Reporting one seed. Collecting more data without a learning curve. Analysing only aggregate metrics, not slices.

What to do next. Day 142 makes data a first-class artefact: validate it on the way in, and version it so every model can name the exact data it saw.

Day 142: Data validation and dataset versioning

Review today: Day 141 (+1), Day 139 (+3), Day 135 (+7), Day 121 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

In production, most failures are data failures, not model failures. Typical examples: an upstream team renames a column, changes units, sends NaNs, or adds a new label value. The model still returns numbers, just wrong ones. Two defences:

1. Validation: does each batch match a schema? A schema records the expected column count and names, the types, which values may be missing, allowed ranges or categories per feature, and the allowed label set. Check every batch at ingestion and at serving time. Fail loudly rather than predict on garbage. You can infer a schema from the training data (observed min/max with some slack) and then review it by hand. Libraries such as Great Expectations, TensorFlow Data Validation and pandera do this at scale; the idea fits in 20 lines.

What validation *cannot* catch: values that are each plausible but whose distribution has moved. A 10% shift in a measurement stays inside every range. That needs distribution tests: drift detection, Day 145.

2. Versioning: which exact data trained this model? Code has git; data needs the same. DVC's core idea (see its docs) is **content addressing**: - Hash the bytes of the data. - Store the file under its hash in a cache or remote. - Commit only the small hash pointer to git.

Identical data gets an identical hash, so it is stored once (deduplication), and any change produces a new version. A log of (hash, parent, message) entries gives you **lineage**: model -> data version -> parent version.

Design choices that matter: - **Canonicalisation.** Hash a canonical serialisation (fixed dtype, fixed column order). Otherwise the same data saved two ways gets two hashes. - **Order.** Should a row-shuffled copy count as the same dataset? For a training set, usually yes, so you can sort rows before hashing to get an order-invariant hash. For a time series, no. - **Sensitivity.** A cryptographic hash changes if a single bit changes. That is correct for provenance, but it means "new version" does not imply "meaningfully different". Pair versions with summary statistics so a human can see what changed.

Block 2: Code walk-through (1h)

`day142_validate_version.py` infers a schema from the training split: 30 columns, finite values, per-feature range widened by 50% of the span, labels {0,1}. It runs six test batches through `validate()`. Then it implements `commit()`: - Canonicalise `X` as float64 C-order bytes plus `y` as int64 bytes, optionally sorting rows first. - Hash with SHA-256, keep the first 12 hex characters, write the data to `store/<hash>.npz` only if it is new, and append a JSON line with hash, rows, message and parent.

Verified (out/day142_validate_version.log): - Clean test batch: PASS. NaNs in 5 rows: FAIL ("5 rows with NaN"). Area x100 (unit change): FAIL, with 143 values outside the allowed range. Dropped column: FAIL ("expected 30 columns, got 29"). Label 2: FAIL. **Radius +10% (silent shift): PASS.** Validation misses it. - v1 = `9f2384ac7409`. Re-committing the same data gives the same hash and stores nothing new (dedup). - Row-shuffled copy: the raw hash differs, but the order-invariant hash is equal. - One label flipped -> v2 `8d7d3315955f`, with lineage `8d7d3315955f <- 9f2384ac7409`. - One value changed by +1e-4 -> the hash differs. - Store: 3 log entries, 211.6 kB on disk.

Day 147's service reports `data_version 9f2384ac7409` on `/health`, so any prediction can be traced back to its training data.

Block 3: Exercises (1h30)

1. Add per-column dtype checks and a max-missing-fraction rule; write a batch that violates each.
2. Tighten the range slack from 50% to 0% and run the clean test batch. How many false failures do you get? What does that say about inferring schemas from small samples?
3. Store diffs: for v2, print which rows and labels changed relative to its parent.
4. Make `commit()` chunk the data (for example 100 rows per chunk), so that a one-row change stores one new chunk instead of a full copy. Measure the disk saving.

Block 4: Self-check (30 min)

1. Name a fault that schema validation cannot catch. *A plausible-looking distribution shift, such as radius +10%. It passed.*
2. Why does dedup fall out of content addressing for free? *Identical bytes give an identical hash, so the file already exists under that name.*
3. When is an order-invariant hash wrong? *When order carries meaning, as in time series or ranked lists.*
4. A +1e-4 change made a new version. Is that a bug? *No. It is correct provenance, but you need summary diffs to judge whether it matters.*
5. What should a model artefact record about its data? *The data-version hash, and ideally the schema version.*

Block 5: Reflection (15 min)

Recall a time a pipeline you used broke because of data. Which check would have caught it: schema, range, drift, or none?

Extra notes. Validate at *both* training and serving. Use the same schema object at both ends, so a silent change at one end does not create training/serving skew.

Going deeper. Read DVC's "get started" docs to see how pointer files and remotes implement today's idea with git. Compare it with the 30-line `commit()` here.

Interview-style question (15 min). "Predictions went bad overnight but the model didn't change. How do you debug?" Check the data version and the validation logs of recent batches. Diff feature statistics against training (drift). Check upstream schema changes and unit changes. Make sure the serving-time feature code matches training. Roll back to the last good data if needed.

Common mistakes. Hashing pickles, which are not canonical. Versioning code but not data. Schemas inferred and never reviewed. Validating only at training time. Treating "PASS" as "no problem".

What to do next. Day 143 does the same for experiments: track every run's parameters, metrics and environment, and make training reproducible down to the weight hash.

Day 143: Experiment tracking and reproducibility

Review today: Day 142 (+1), Day 140 (+3), Day 136 (+7), Day 122 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

After 20 experiments, nobody remembers which learning rate produced the good number. An **experiment tracker** records for each run: - the **parameters** (hyperparameters and the data version); - the **metrics** (validation scores, possibly per epoch); - the **artefacts** (model file, plots); - the **environment** (library versions, git commit, hardware).

You can then query the runs ("mean validation accuracy by config"). MLflow is the common open-source choice: runs grouped in experiments, logged with `log_params`/`log_metrics`, and queried with `search_runs` into a pandas table. A tracker can be as small as a JSON-lines file. The discipline matters more than the tool.

Selection hygiene. Choose configurations by *mean over seeds* on validation data. Touch the test set once, at the end. Picking the single best run out of many is a maximum over noise, and it is optimistic.

Reproducibility levels: 1. Same process, same seed -> bit-identical weights. You need to seed every RNG: Python, numpy, torch, and the data-split RNG. 2. Fresh process -> same weights. You also need fixed library versions and no hidden state. 3. Different thread count or hardware -> same weights. Not guaranteed: parallel floating-point reductions may run in a different order. GPUs add non-deterministic kernels unless you enable deterministic algorithms (at some speed cost). 4. Different machine or library version -> usually close, not identical.

Verify reproducibility by **hashing the trained weights**, not by eyeballing accuracy.

Block 2: Code walk-through (1h)

`day143_tracking_repro.py` runs a grid: hidden size {8, 32, 128} x lr {1e-3, 1e-2} x 3 seeds, scored on a validation split carved from training data. It logs every run to both MLflow (file store) and a small own JSON-lines tracker. The MLflow setup needed `MLFLOW_ALLOW_FILE_STORE=true`, because version 3.17 refuses the file store by default, and sqlite needs SQLAlchemy, which is

not in mlflow-skinny. The script then retrains the chosen config, reports test accuracy once, and hashes the weights (`_mlops.weights_hash`, SHA-256 over all parameter bytes) across seeds, processes and thread counts.

Verified (out/day143_tracking_repro.log): - 18 runs logged to both trackers in 4.4 s. - Mean +/- std of validation accuracy over 3 seeds:

h	lr	mean	std
128	0.001	0.972	0.000
32	0.01	0.972	0.000
128	0.01	0.969	0.005
32	0.001	0.969	0.005
8	0.001	0.969	0.005
8	0.01	0.963	0.009

- Own tracker: the single best run is h=8 lr=0.001 seed 0 (val 0.972), but that config's mean is 0.969, so the "best run" is a lucky seed.
- Chosen by mean: h=128 lr=0.001. **Test accuracy 0.951**, reported once. It is lower than validation; this is selection optimism plus small-sample noise.
- Same seed twice: weights `53df326a18b85c38` both times, equal. Seed 1: `61b59b2a992a880a`, with test accuracy 0.972 vs 0.965 for seed 0. The seed alone moves test accuracy by one example.
- Fresh process with 1 thread and with 4 threads: both `53df326a18b85c38`. On this small CPU model, thread count did not change the result; that is not guaranteed in general.
- Environment logged: python 3.13.5, torch 2.15.0+cpu, numpy 2.5.4, mlflow 3.17.2.

The Day 147 project uses the tied-best h=32 lr=1e-2 seed-0 config: weights `53df326a18b85c38`, test 0.965.

Block 3: Exercises (1h30)

1. Add per-epoch validation loss with `mlflow.log_metric(step=...)` and plot two runs.
2. Log the Day 142 data hash as a parameter, so `search_runs` can filter by data version.
3. Set `torch.use_deterministic_algorithms(True)` and re-run. Does anything change on CPU?
4. Add 5 more seeds to the two tied configs. Does the tie break?

Block 4: Self-check (30 min)

1. Why choose by mean over seeds? *A single best run is a maximum over noise. Here h=8 "won" a single run but has mean 0.969.*
2. Why is test accuracy (0.951) below validation (0.972)? *Selection on validation is optimistic, and 143 test rows are noisy.*
3. How do you prove a run is reproducible? *Hash the weights. Equal hashes mean bit-identical models.*
4. Will 1 vs 4 threads always give equal weights? *No. Reduction order can change floating-point results. Here it happened to match.*
5. What does mlflow-skinny lack that blocked sqlite? *SQLAlchemy.*

Block 5: Reflection (15 min)

Could you re-create your best Week 20 result from its log alone today? List what is missing.

Extra notes. Record the git commit and a `pip freeze` with every run. "It worked last month" is usually a library-upgrade story.

Going deeper. Read MLflow's tracking and model-registry docs. A registry adds named versions and stages (staging, production) on top of the runs we logged; Day 146's canary is how a version moves between stages.

Interview-style question (15 min). "A colleague reports 98% from one run. What do you ask?" How many runs and seeds were tried? Mean and spread? Was the test set used for selection? Which data version and environment? Can it be reproduced from the logged parameters?

Common mistakes. Seeding torch but not the data split. Tuning on test. Reporting the max over seeds. Not logging library versions. Assuming GPU runs are deterministic.

What to do next. Day 144 packages the model: export to ONNX, serve over HTTP, and measure latency under load.

Day 144: Packaging and serving: ONNX, onnxruntime, FastAPI and a load test

Review today: Day 143 (+1), Day 141 (+3), Day 137 (+7), Day 123 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Packaging turns "weights in a notebook" into a versioned artefact with a fixed interface. The artefact holds: - the model; - its preprocessing (we keep normalisation *inside* the model to prevent skew); - metadata: model version, data version, threshold, schema.

ONNX is an open graph format for models. `torch.onnx.export` traces the forward pass into a graph of standard operators, with a dynamic batch axis declared. **onnxruntime** then runs that graph without PyTorch: a smaller dependency and often lower per-call overhead, because there is no Python autograd machinery around each op. Always verify an export numerically against the original model. Unsupported ops or dynamic control flow can export wrongly or not at all.

Serving patterns: - *Online*: an HTTP request per prediction, latency-sensitive. - *Batch*: score a table offline, throughput-sensitive, and much cheaper (Day 146). - *Streaming*: in between.

For online serving, FastAPI with pydantic models gives typed request validation, so malformed JSON gets a 422 automatically, and uvicorn runs it.

Load testing. Measure *tail* latency (p95/p99), not just the mean, at realistic concurrency. Also measure throughput (requests/s and rows/s). Little's law ties them together: concurrency ~= throughput x latency. Once the server saturates, adding concurrency only grows the queue, so latency rises while throughput stays flat. Batching more rows per request raises rows/s but raises per-request latency.

Block 2: Code walk-through (1h)

`day144_onnx_serving.py`: - exports the seed-0 MLP with a dynamic batch axis; - checks it with `onnx.checker`; - compares logits on all 143 test rows; - times torch vs onnxruntime per call at batch 1, 64 and 1024 (one thread each).

It then starts `serve_app.py` under uvicorn (1 worker, localhost), once per backend, and drives it with an `httpx` async client at concurrency 1 and 16.

Verified (out/day144_onnx_serving.log): - Export is 9622 bytes; checker OK; max |logit difference| on 143 rows is 0.00e+00; predictions identical. - Per-call time (1 thread): batch 1, torch 21.5 µs vs ORT 5.1 µs (4.2x); batch 64, 23.8 vs 9.7 µs (2.4x); batch 1024, 80.6 vs 75.1 µs (1.1x). The ONNX advantage is fixed per-call overhead, so it fades when real compute dominates. - HTTP, ONNX backend: c1 x1 row p50 1.4 / p95 2.1 / p99 2.2 ms, 650 req/s; c16 x1 row p50 17.7 / p95 52.5 / p99 77.9 ms, 705 req/s; c16 x64 rows 390 req/s = 24,991 rows/s (p95 135.8 ms). - HTTP, torch backend: c1 567 req/s; c16 678 req/s (p95 55.1 ms); c16 x64 rows 402 req/s = 25,697 rows/s (p95 78.2 ms).

Readings: - At the HTTP level the 4x model speed-up nearly vanishes: 650 vs 567 req/s at c1. The model takes about 5-20 µs of a roughly 1.5 ms request, and the rest is HTTP, JSON and Python. **Profile before optimising the model.** - Going from c1 to c16 barely raises throughput (650 -> 705) but multiplies p50 by about 13x: the server is saturated and requests queue. - With 64 rows per request, rows/s rise about 35x while requests/s fall. The two backends are similar (25.7k vs 25.0k rows/s). The ONNX run's higher p95 at that setting is within the noise of a client sharing the CPU, so I would not call it a real difference without repeats. - **A flaw found:** a wrong-shape input returned HTTP **200** with an error body, while a wrong type gave a proper 422. Monitoring that counts status codes would miss the shape errors. The Day 147 service fixes this.

Caveat: the client and server share one machine, and there is no network. Real latency adds network round-trips; real throughput would come from more workers or cores.

Block 3: Exercises (1h30)

1. Run uvicorn with `--workers 2` and repeat c16. Does throughput scale? Remember the client also needs CPU.
2. Add a `/predict_batch` that accepts up to 512 rows and find the rows-per-request that maximises rows/s while keeping p95 < 50 ms.
3. Set onnxruntime's `intra_op_num_threads` to 4 and re-time batch 1024.
4. Export with a fixed batch size and try to send batch 2. What error do you get?

Block 4: Self-check (30 min)

1. Why did ORT's 4.2x win shrink to 1.1x at batch 1024? *It saves fixed overhead per call, and at large batches arithmetic dominates.*
2. Why was the HTTP gain so small? *The model is about 1% of request time; HTTP and JSON dominate.*
3. What does p50 rising 13x with flat throughput mean? *Saturation: extra concurrency only queues.*
4. Why put normalisation in the model? *So the serving code cannot skip or mismatch it (training/serving skew).*
5. What was wrong with the shape-error response? *It returned HTTP 200, so clients and monitors treat it as success. It should be a 4xx.*

Block 5: Reflection (15 min)

For a model you built earlier in the course, what fraction of request latency would be the model itself? Guess, then plan how you'd measure it.

Extra notes. For big neural models the picture flips: the model dominates, and GPU batching (dynamic batching servers) is the main lever. For this tiny MLP the framework is the bottleneck.

Going deeper. Read the ONNX Runtime docs on graph optimisation levels and execution providers, and FastAPI's docs on request bodies and validation.

Interview-style question (15 min). "p99 latency doubled after a deploy, but the model is the same. Where do you look?" Request payload size, serialisation, worker count, CPU contention, garbage-collection pauses, a dependency upgrade, logging added to the hot path, and queueing. Compare the model's own timer against end-to-end latency to split the time.

Common mistakes. Reporting mean latency only. Load testing from the same box and calling it production numbers. Not checking export parity. Returning 200 on errors. Assuming a faster runtime speeds up the endpoint.

What to do next. Day 145 watches the model after launch: drift tests on the inputs, and a loop to find label errors in the data.

Day 145: Monitoring, drift detection (PSI and KS) and the label-error loop

Review today: Day 144 (+1), Day 142 (+3), Day 138 (+7), Day 124 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

After launch you usually do not get labels immediately, so you monitor *proxies*. What to watch: - **Service health:** errors, latency, rejected inputs. - **Input drift:** has P(x) moved away from the training data? - **Output drift:** has the distribution of predictions moved? - **Performance**, once delayed labels arrive.

Kinds of shift: - *Covariate shift* is a change in P(x). - *Prior or label shift* is a change in P(y). Inputs move because the class mix moved. - *Concept drift* is a change in P(y|x). It is the dangerous one, and it is invisible to input monitoring.

PSI (population stability index). - Bin a feature using reference-data quantiles (deciles here). - Compare the fraction of data in each bin, reference e vs current a: PSI = Sum (a - e).ln(a/e). - Common rules of thumb are < 0.1 stable, 0.1-0.2 moderate, > 0.2 large. These numbers ignore sample size and how many features you test, which today's run shows matters.

Two-sample KS test. The statistic is the maximum gap between the two empirical CDFs, and it comes with a p-value. Testing 30 features at $p < 0.01$ each gives many chances of a false alarm. **Bonferroni** ($p < 0.01/30$) controls the family-wise rate.

Drift != degradation. A shifted input may lie in a region the model handles fine. Alert on drift to *investigate*; judge harm with labels.

Data-centric error loop. Label noise is common in real datasets. Train with cross-validation, and flag training rows where the *out-of-fold* model gives the given label very low probability. Out-of-fold matters: a model that saw a row will just memorise its wrong label. Then have a human review the flagged rows, fix or drop them, retrain, and measure. This is the idea behind confident-learning tools such as cleanlab, done by hand here.

Block 2: Code walk-through (1h)

`day145_drift_labels.py` implements PSI (decile bins from the reference, infinite outer edges) and its own KS statistic, checked against `scipy.stats.ks_2samp`. Reference = training split. Each "production" batch is 200 rows resampled from the test split, with an injected fault. Accuracy is measured on each batch because we know its labels. Then it flips 5% of training labels, gets 5-fold out-of-fold probabilities, flags rows whose given label has out-of-fold probability < 0.2, and retrains without them.

Verified (out/day145_drift_labels.log): - Own KS statistic 0.0859 = scipy's 0.0859 (train vs test "mean radius").

batch (200 rows)	PSI>0.2 (of 30)	KS p<0.01	KS Bonferroni	accuracy
no drift	2 (max 0.31)	1	0	0.945
texture +1 sd	6 (max 2.16)	1	1	0.985
area x1.3	6 (max 1.27)	4	3	0.945
prior shift, 80% malignant	24 (max 0.84)	26	24	0.965
noise +0.3 sd, all features	1 (max 0.24)	2	0	0.965

- False alarms over 50 no-drift batches (any feature flagged): PSI > 0.2 **48/50**; KS p < 0.01 26/50; KS Bonferroni 2/50.
- Output PSI of the predicted probability: 0.79 under prior shift, 0.05 with no drift.
- Label loop: 21 flips; 37 flagged, 20 of them true flips (precision 0.54, recall 0.95). Test accuracy: clean labels 0.965, noisy labels 0.909, noisy minus flagged rows 0.958.

Readings: - The PSI > 0.2 rule fires on 48 of 50 *clean* batches. With 30 features and 200 rows, sampling noise alone pushes some feature over 0.2. That is why Day 147 calibrates the threshold on no-drift windows instead. - Bonferroni-KS is quiet on clean data (2/50) and still catches texture, area and prior shift. Both tests nearly miss the noise injection. - **No injected drift lowered accuracy.** Texture +1 sd even *raised* it, to 0.985, on this batch. Drift alerts are a reason to look, not proof of harm. - Prior shift lights up almost every feature and the output distribution, a signature that differs from a single-feature sensor fault (area). - Label noise cost 5.6 points of test accuracy, and removing flagged rows recovered almost all of it: 0.958 vs 0.965. That is despite dropping 17 correctly labelled rows. In practice, send flagged rows to review instead of deleting them.

Block 3: Exercises (1h30)

1. Plot no-drift PSI against batch size (50, 200, 1000). How should the threshold scale with window size?
2. Inject *concept drift*: flip the label rule for large-area rows while keeping x unchanged. Which monitor notices? (Only labelled performance can.)
3. Replace the 0.2 cut-off on out-of-fold probability with 0.1 and 0.3. Plot precision against recall.
4. Use output PSI alone as the monitor. Which injected faults does it catch?

Block 4: Self-check (30 min)

1. Why did PSI > 0.2 alarm 48/50 times on clean data? *Thirty features times sampling noise in 200-row batches: the threshold ignores multiplicity and sample size.*
2. What does Bonferroni do? *It divides the significance level by the number of tests to control the chance of any false alarm.*
3. Did drift imply worse accuracy here? *No. Accuracy stayed at or above the no-drift 0.945 in every case.*
4. Why use out-of-fold predictions to find label errors? *An in-sample model memorises the wrong labels.*
5. Which shift can input monitoring never see? *Concept drift, a change in P(y|x) with P(x) unchanged.*

Block 5: Reflection (15 min)

For a model you care about, how long would it take for labels to arrive after a prediction? What would you monitor in that gap?

Extra notes. Monitor the *same* features the model uses, computed by the *same* code. Keep the reference data versioned (Day 142) so "drift relative to what" is answerable.

Going deeper. Compare PSI with Jensen-Shannon distance and with a learned "domain classifier" that tries to tell reference rows from current rows. Its AUC is a multivariate drift score.

Interview-style question (15 min). "Your drift dashboard is always red and people ignore it. Fix it." Calibrate thresholds on historical no-drift windows. Correct for the number of features. Alert on sustained drift in important features. Pair drift with output and performance signals. Route alerts to investigation runbooks rather than pages.

Common mistakes. Using PSI rule-of-thumb thresholds blindly. Testing many features without correction. Equating drift with degradation. Finding label errors in-sample. Auto-deleting flagged rows.

What to do next. Day 146 covers changing models safely (A/B tests and canaries), what serving costs, and how CI and containers fit, with untested example files.

Day 146: A/B tests, canary rollouts, cost and latency budgets, Docker and CI

Review today: Day 145 (+1), Day 143 (+3), Day 139 (+7), Day 125 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Offline wins are not online wins. A new model that is better on the test set can be worse for users: different traffic, feedback loops, latency. Two rollout tools:

- **A/B test.** Randomly split users between model A and model B and compare a business metric, such as conversion.
- Fix the sample size *in advance* from the smallest effect worth detecting. The required n grows like 1/effect^2, so halving the effect needs about 4x the users.
- Test once at the end. **Peeking** (checking repeatedly and stopping at the first p < 0.05) inflates false positives. If you must monitor continuously, use sequential methods designed for it.
- **Canary.** Send a small slice of traffic (here 5%) to the new model and watch guardrail metrics: errors, latency, complaint rate. Promote step by step if nothing is significantly worse; roll back automatically if it is. A canary protects against *bad* releases. It is not powered to prove small *improvements*.
- **Shadow mode** runs the new model on real traffic without acting on its outputs, so you can compare predictions and latency at zero user risk.

Cost and latency budgets. Start from requirements (peak requests/s, p95 SLO, predictions/day). Measure throughput per core, size the fleet with headroom (we target 60% utilisation), and price it. Ask whether online serving is needed at all: if predictions can be precomputed, batch scoring is often orders of magnitude cheaper.

Docker and CI concepts: - A **container image** packages the OS libraries, Python, pinned dependencies, code and artefacts, so "works on my machine" becomes "works in this image". Practices: pin versions, use a slim base image, run as a non-root user, add a health check, and keep artefacts versioned (or pulled by hash at start-up). - **CI** runs on every push. For ML it should cover unit tests, data-validation tests, *golden-prediction* tests (fixed inputs, expected outputs), a metric floor (block release if accuracy falls below X), and building the image. - **CD** then deploys the image through canary to production.

Block 2: Code walk-through (1h)

`day146_ab_canary_cost.py`: - computes per-arm sample sizes with the normal approximation (alpha 0.05, power 0.8); - simulates 2000 A/A experiments, tested once and with 10 peeks, plus A/B experiments at 10% -> 11%, all with a two-proportion z-test; - simulates a canary rule: after each 1000 canary requests, roll back if a one-sided z-test says the canary's error rate is worse at p < 0.01, up to 10 steps; two random seeds per scenario; - builds a cost table from Day 144's measured numbers plus **assumed** workload and price.

Verified (out/day146_ab_canary_cost.log): - Users per arm at a 10% baseline: 3841 to detect 12%; 14,751 for 11%; 57,763 for 10.5%. Halving the effect needs about 4x the users. - 2000 simulated experiments at n = 14,751 per arm: A/A tested once, false-positive rate 0.046 (nominal 0.05); **A/A with 10 peeks, 0.191**; true 10% -> 11% tested once, power 0.791 (designed 0.8). - Canary, both seeds: equal error rates (0.035) -> PROMOTED after 10,000 canary requests; 0.045 vs 0.035 -> ROLLED BACK after 6000; 0.07 vs 0.035 -> ROLLED BACK after 1000. - Cost. Measured: 650 req/s, p95 52.5 ms at c16, 24,991 rows/s batched. **Assumed** (not measured, not a quoted price): peak 200 req/s, 2,000,000 predictions/day, $0.05 per core-hour. Online at 60% utilisation: 1 core, $1.20/day. Batch scoring the same 2M rows: 1.3 core-minutes, $0.0011/day.

Readings: - Peeking quadrupled the false-positive rate, from 0.046 to 0.191. - The canary catches a 2x error-rate jump at the first check but needs 6000 requests for a 0.035 -> 0.045 regression. Small regressions take traffic to detect. - Batch was about 1000x cheaper than online here. For this model, the cost is the always-on core, not the compute. Note also that the measured c16 p95 of 52.5 ms already misses a 50 ms SLO on this shared-CPU test, so the "1 core" sizing needs a real load test on the target hardware.

Untested files (no Docker on this box): `project/Dockerfile` (python:3.13-slim, pinned `requirements-serve.txt`, non-root user, HEALTHCHECK on `/health`, uvicorn 1 worker) and `project/ci/github-actions-ci.yml` (checkout, set up Python 3.13, install, run `day147_project.py` and pytest, upload the model card). Both start with the comment **"UNTESTED"**. They show the

shape of the setup; I have not built or run either, and they may need fixes.

Block 3: Exercises (1h30)

1. Re-run the peeking simulation with 3 and 30 peeks. Plot false-positive rate against the number of peeks.
2. Change the canary to 20% traffic. How much sooner is the 0.045 regression caught?
3. Redo the cost table with your own cloud's real price per core-hour, and with a 3-replica minimum for availability.
4. Read the Dockerfile reference and list two things that could break this untested Dockerfile. (Hint: the build context and the paths in COPY.)

Block 4: Self-check (30 min)

1. Why does detecting 10% -> 10.5% need about 15x the users of 10% -> 12%? *n ? 1/effect^2, and the effect is 4x smaller, so about 16x: 57,763 vs 3841.*
2. What did peeking do? *It raised the A/A false-positive rate from 0.046 to 0.191.*
3. What is a canary for? *Catching harmful releases early with limited exposure, not proving small gains.*
4. Why is batch so much cheaper here? *You pay only for compute minutes, not an always-on core sized for peak.*
5. What makes the Docker/CI files trustworthy? *Nothing yet. They are untested until built and run in CI.*

Block 5: Reflection (15 min)

Which of your course models genuinely needs online predictions, and which could be precomputed nightly?

Extra notes. Guardrail metrics (latency, errors, crash rate) should be able to roll back a canary even when the business metric looks good.

Going deeper. Read the GitHub Actions docs on workflow syntax and caching, and compare the untested YAML against them.

Interview-style question (15 min). "The new model wins offline by 1 point. How do you launch it?" Shadow it to compare predictions and latency, then canary with guardrails and automatic rollback. If the business impact matters, run a pre-registered A/B test sized for the minimum effect worth detecting. Don't peek; document it; keep the old model ready for rollback.

Common mistakes. Peeking. Underpowered tests. Canaries without automatic rollback. Costing at average load instead of peak. Calling untested Dockerfiles "production-ready".

What to do next. Day 147 assembles it all into one production-style service with tests, monitoring, a load test and a model card.

Day 147: Weekly project: a production-style model service

Review today: Day 146 (+1), Day 144 (+3), Day 140 (+7), Day 126 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The project joins the week's pieces into one small service that a reviewer could actually operate: 1. **Traceable artefacts.** An ONNX model plus `meta.json`, which records the model weight hash, data-version hash (Day 142), threshold (Day 141's cost choice), feature names, drift reference and calibrated drift threshold, and golden predictions. 2. **A strict interface.** Wrong shapes, NaNs, wrong types and oversized batches get HTTP **422**, fixing the Day 144 flaw where a bad shape returned 200. 3. **Tests as a release gate:** health, golden-prediction parity, an accuracy floor, bad inputs, a batch limit, and a drift alarm that must stay quiet on clean data and fire on drifted data. 4. **Monitoring built in.** `/metrics` reports request, row and rejection counts, model-only p95 latency, and drift (max PSI over a rolling 500-input window, which feature, and whether it alerts). 5. **A calibrated drift threshold.** Day 145 showed PSI > 0.2 alarms on 48/50 clean batches. Instead, take the 99th percentile of max-PSI over no-drift windows. 6. **A load test against an SLO** (p95 < 50 ms on this box). 7. **A model card** generated from the same run, so its numbers cannot drift from reality. 8. **Container and CI files**, clearly labelled UNTESTED.

Model cards (from Mitchell et al.'s proposal; written here in my own structure) document what a model is, the data it used, intended and out-of-scope uses, metrics with uncertainty and by slice, how it is monitored, and known limits. Writing the card *from the run* rather than by hand keeps it honest.

Block 2: Code walk-through (1h)

`day147_project.py`: - trains the Day 143 tied-best default config (h=32, lr=1e-2, seed 0), exports it to ONNX, and writes `project/artifacts/`; - calibrates the drift threshold on 200 no-drift windows of 143 rows resampled from the training data; - runs `pytest` on `project/test_service.py`; - starts `project/service.py` under uvicorn, runs a 500-request load test at concurrency 8, and sends clean traffic then area x1.3 traffic, reading `/metrics` after each; - writes `project/MODEL_CARD.md`.

Two bugs found and fixed while building it: - An `exec` of the service file inside the build script overwrote its artefact path, causing FileNotFoundError on `meta.json`. I removed the exec. - A test sending `float("nan")` crashed *inside the test client*, because strict JSON cannot encode NaN. Python's JSON parser *does* accept the literal `NaN`, though, so a real client can send it. The test now posts the raw text `NaN` and checks for a 422.

Verified (out/day147_project.log): - Data version `9f2384ac7409`; model `53df326a18b85c38`; test accuracy 0.965; threshold 0.8. At 0.8: missed malignant 2, false alarms 4; accuracy by mean-area tertile small 1.000, medium 0.915, large 0.958. Day 141's large-tertile 0.979 was at threshold 0.5; the higher threshold trades accuracy for fewer misses. - Calibrated drift threshold: **0.234** (99th percentile of max-PSI over 30 features), vs the 0.2 rule of thumb. - pytest: **10 passed**. - Load test, 500 single-row requests at concurrency 8: p50 7.8 ms, p95 21.7 ms, 769 req/s. SLO p95 < 50 ms met. Model-only p95 about 0.17 ms, so roughly 99% of request time is not the model. - `/metrics` after 500 clean rows: max PSI 0.187 ("worst symmetry"), alert false, rejected 0. - `/metrics` after 500 rows with area x1.3: max PSI 0.658 ("mean area"), **alert true**. It names the right feature.

Honest limits: - The threshold was calibrated on 143-row windows while the service uses 500-row windows. Larger windows have less sampling noise, so the threshold is conservative there (fewer false alarms, slower detection). - Labels are not observed, so the service cannot see accuracy loss or concept drift. - One seed; 143 test rows, where one error is 0.007. - Docker and CI are untested, and the load test ran client and server on one machine.

Block 3: Exercises (1h30)

1. Recalibrate the drift threshold with 500-row windows to match the service, and re-run the quiet and loud tests.
2. Add a `/feedback` endpoint that accepts delayed labels and reports rolling accuracy. Then inject concept drift and watch it.
3. Add a shadow model (Day 143's h=128 config) that scores every request without affecting the response. Log disagreement rate and latency.
4. Make the CI workflow fail if `MODEL_CARD.md` changes without a version bump.

Block 4: Self-check (30 min)

1. Why generate the model card from the run? *So every number in it is the measured one, and it cannot go stale.*
2. What do golden-prediction tests catch? *Export, preprocessing or dependency changes that alter outputs.*
3. Why calibrate the PSI threshold? *A fixed 0.2 gave 48/50 false alarms with 30 features. Calibration gives 0.234 at a 1% false-alarm target per window.*
4. Did the drift alert identify the cause? *Yes. Max PSI 0.658 on "mean area" after area x1.3.*
5. What can this service still not detect? *Concept drift and accuracy loss, without labels.*

Block 5: Reflection (15 min)

If you were on call for this service, which single `/metrics` field would you page on, and which would you only chart?

Extra notes. Real services add authentication, rate limiting, structured logs, tracing, multiple replicas and a registry-driven deploy. Each is a layer on the same skeleton.

Going deeper. Build the Dockerfile on a machine that has Docker, fix whatever breaks, and record what you changed. Until then it stays labelled UNTESTED.

Interview-style question (15 min). "Walk me through what happens when this model is retrained." New data is validated and versioned. Training is tracked and reproducible, with selection by mean over seeds. CI exports, checks parity and runs golden, floor and input tests. A new card is generated. Then shadow, canary with rollback, and promotion, with the drift reference updated to the new training data.

Common mistakes. Hand-written model cards. Thresholds copied from blog posts. Returning 200 on errors. Tests that only run on the happy path. Calling localhost load tests production numbers.

What to do next. Week 22 continues Month 5. LLMOps (originally scheduled for Day 143) and the Month-5 capstone (originally Days 144-147) move later; see the SYLLABUS.

Weekly quiz: Week 21 (12 questions; 40 minutes; covers Days 141-147)

1. Give the four lifecycle stages and say which one most often changes between iterations.
2. On Day 141, why was threshold 0.8 chosen over 0.5? Give both costs.
3. What did Day 141's learning curve suggest about buying more data?
4. Which Day 142 fault passed schema validation, and what catches it instead?
5. Why did a row-shuffled copy need an order-invariant hash to match?
6. On Day 143, why is "h=8 seed 0, val 0.972" a bad reason to pick h=8?
7. How was reproducibility verified, and did thread count matter here?
8. Why did onnxruntime's per-call speed-up (4.2x at batch 1) barely show at the HTTP level?
9. What was the false-alarm rate of PSI > 0.2 over 30 features on clean batches, and of Bonferroni-KS?
10. Did any injected drift reduce accuracy on Day 145? What does that imply for alerting?
11. What did peeking 10 times do to the A/A false-positive rate?
12. In the project, what drift threshold was used, how was it obtained, and what did `/metrics` show after area x1.3?

Answers

1. Scoping, data, modelling, deployment/monitoring. The data usually changes the most.
2. Cost = 10 x missed + false alarms: 24 at 0.8 vs 32 at 0.5 (missed 2 vs 3, false alarms 4 vs 2).
3. It was nearly flat after about 25% of the data (0.946 -> 0.953), so more of the same data is a poor investment.
4. Radius +10% passed. Drift detection (PSI/KS) catches distribution shifts.
5. Raw bytes depend on row order. Sorting rows before hashing makes the hash order-invariant.
6. It is a maximum over noise. That config's mean over seeds was 0.969; pick by mean.
7. By hashing the weights: same seed gave equal hashes (`53df326a18b85c38`), and fresh processes with 1 and 4 threads gave the same hash. Thread count did not matter here, but that is not guaranteed.
8. The model is about 5-20 µs of a roughly 1.5 ms request; HTTP, JSON and Python dominate (650 vs 567 req/s).
9. PSI: 48/50 batches alarmed. KS with Bonferroni: 2/50.
10. No. Accuracy stayed at or above 0.945 in every case. Drift alerts should trigger investigation, not be read as degradation.
11. It rose from 0.046 (test once) to 0.191.
12. 0.234, the 99th percentile of max-PSI over 200 no-drift windows. After area x1.3: max PSI 0.658 on "mean area", alert true; after clean traffic, 0.187 with no alert.